

Design Document

Airplane Auto Landing System

Course: ECE163 Introduction to Small-Scale UAV Theory and Practice

Instructor: Gabriel Hugh Elkaim

Team: Autonomous Landing Team

Team Members

Xianle Zeng

Gavin Wang

Hanwen Chen

March 18, 2026

Contents

1	Introduction	4
1.1	Project Background	4
1.2	Problem Statement	4
1.3	Project Goals	5
1.3.1	Essential Goals	5
1.3.2	Stretch Goals	5
1.4	Success Metrics	5
2	Overview	6
2.1	High-Level System Description	6
2.2	System Architecture	7
2.3	Module Summary	8
2.4	How the Code Will Be Run	8
2.5	Expected Behavior	9
3	Module Details	9
3.1	Module A: LQR Controller	9
3.1.1	Purpose	9
3.1.2	Inputs	10
3.1.3	Outputs	11
3.1.4	Internal States / Attributes	11
3.1.5	Main Methods	11
3.1.6	Logic / Algorithm	12
3.1.7	Equations (if applicable)	12
3.1.8	Integration with Existing Framework	13
3.2	Module B: State Estimator	13
3.2.1	Purpose	13
3.2.2	Inputs	13

3.2.3	Outputs	14
3.2.4	Internal States / Attributes	14
3.2.5	Main Methods	14
3.2.6	Logic / Algorithm	15
3.2.7	Integration Notes	15
3.3	Module C: Landing Guidance (AutoLandGuidance)	16
3.3.1	Purpose	16
3.3.2	Inputs	16
3.3.3	Outputs	16
3.3.4	Internal States / Attributes	17
3.3.5	Main Methods	17
3.3.6	Logic / Algorithm	17
3.3.7	Equations	18
3.3.8	Reference Trajectory	19
3.3.9	Integration with Existing Framework	19
4	Development Plan	19
4.1	Task Breakdown	19
5	Testing Strategy	20
5.1	Testing Philosophy	20
5.2	Incremental Testing Plan	20
5.3	Evaluation Scenarios	21
5.3.1	Scenario 1: Nominal Case	21
5.3.2	Scenario 2: Disturbance Case	21
5.3.3	Scenario 3: Stress Test	21
5.4	Planned Outputs	22
5.5	Final Success Criteria	22
6	Conclusion	22

1. Introduction

1.1 Project Background

Unmanned aerial vehicles (UAVs) rely on accurate state estimation, robust control systems, and reliable guidance algorithms in order to operate autonomously. Modern UAV autopilots must be able to stabilize the aircraft, follow certain trajectories, and safely complete flight phases such as landing. Most simulated environments assume perfect knowledge of the aircraft state. However, in reality, the UAV systems have noisy sensor measurements which must be filtered before sending it to the Autopilot to control the aircraft.

The baseline UAV simulator used in this project provides a physics-based aircraft model, a PID-based autopilot controller, and a perfunctory state estimator with complementary filters. While this allows the aircraft to maintain stable flight and track basic commands, several important capabilities are missing. In particular, the simulator is not as robust as we would want and does not include an autonomous landing guidance system that most modern commercial flights use, such as an Instrument Landing System (ILS). We want to implement alternative control strategies such as optimal control, or realistic state estimation methods that account for sensor noise.

This project extends the baseline simulator by introducing three key components. First, a landing-guidance module is developed to generate reference commands that guide the aircraft through the approach, glide slope, flare, and touchdown phases of landing. Second, a state-space Linear Quadratic Regulator (LQR) controller is implemented as an alternative to the existing PID controller, enabling a comparison between classical and optimal control approaches. Finally, a Kalman filter-based state estimation system is introduced to estimate the aircraft states from noisy sensor measurements.

Together, these additions enhance the simulator's realism and autonomy, allowing it to more closely represent the challenges faced in real UAV flight systems. The integration of guidance, control, and estimation also provides a framework for evaluating how different control strategies perform under realistic conditions.

1.2 Problem Statement

The goal of this project is to design and implement a system that enables autonomous landing and improved flight control within the UAV simulation environment. The project focuses on three key areas: trajectory guidance for landing, optimal control using a Linear Quadratic Regulator (LQR), and state estimation using Kalman filtering.

Specifically, the system generates reference commands that guide the UAV through a complete landing sequence, including approach alignment, glide slope descent, flare, and touchdown. These commands are then tracked by the aircraft control system using either the baseline PID controller

or the newly implemented LQR controller. In addition, a Kalman filter is used to estimate the aircraft's states from simulated sensor measurements, providing a more realistic representation of how UAVs operate in practice.

By integrating these components, the project aims to improve the baseline system by adding autonomous landing capability, introducing optimal control methods, and incorporating robust state estimation.

1.3 Project Goals

1.3.1 Essential Goals

- Implement a landing-guidance module that generates reference commands for approach, glide slope descent, flare, and touchdown
- Implement runway GUI and dynamics for landing and better visualization
- Implement a state-space Linear Quadratic Regulator (LQR) controller and integrate it into the UAV simulator.
- Implement a Kalman filter-based state estimator that estimates aircraft states using simulated sensor measurements.
- Compare the performance of the LQR controller with the baseline PID controller in terms of stability, tracking accuracy, and landing performance.

1.3.2 Stretch Goals

- Improve landing performance by incorporating cross-track correction and runway alignment during the approach phase.
- Optimize LQR weighting matrices to improve tracking performance and landing smoothness.
- Evaluate robustness of the control and estimation systems under sensor noise and disturbances such as wind.

1.4 Success Metrics

The success of the project will be evaluated using quantitative performance metrics derived from simulation results.

Proposed metrics:

- Root-mean-square (RMS) altitude-tracking error during the landing segment less than 5 m and RMS course-tracking error less than 8° .
- Touchdown completed without numerical instability, with descent rate reduced during flare and altitude remaining at or above runway level.
- LQR and PID can both be run on the same landing scenario and compared using tracking error, settling behavior, and control effort plots.
- The EKF estimator yields lower RMS error than the complementary-filter baseline for the primary controller states such as altitude, airspeed, roll, pitch, and course.

2. Overview

2.1 High-Level System Description

The project adds a new closed-loop layer on top of the baseline ECE163 simulator, following the same modular style emphasized in the final-project description. The added layer begins with a landing-guidance module that converts the current aircraft position relative to the runway into commanded altitude, airspeed, and course. Those commands are then tracked by either the baseline PID controller or a new trim-centered LQR controller. The controller drives the existing nonlinear aircraft model, which propagates the full rigid-body state and aerodynamic response of the UAV.

To make the simulation more realistic, the plant output is also passed through the sensor model to generate noisy measurements. A state-estimation module consumes those measurements and reconstructs the states needed by the controller. This allows the same landing scenario to be run using truth-state feedback, complementary-filter estimates, or EKF estimates. Around this closed-loop stack, the project includes GUI-based and script-based test harnesses for tuning, comparison, and final evaluation.

The proposed system consists of the following major components:

1. A landing-guidance module responsible for generating runway-aligned reference commands for approach, glide slope, flare, touchdown, and rollout
2. A flight-control layer responsible for tracking those references with either the existing PID controller or the new LQR controller
3. The existing UAV simulation and dynamics framework
4. A sensor, estimation, and testing pipeline used to compare truth-state, complementary-filter, and EKF-based operation

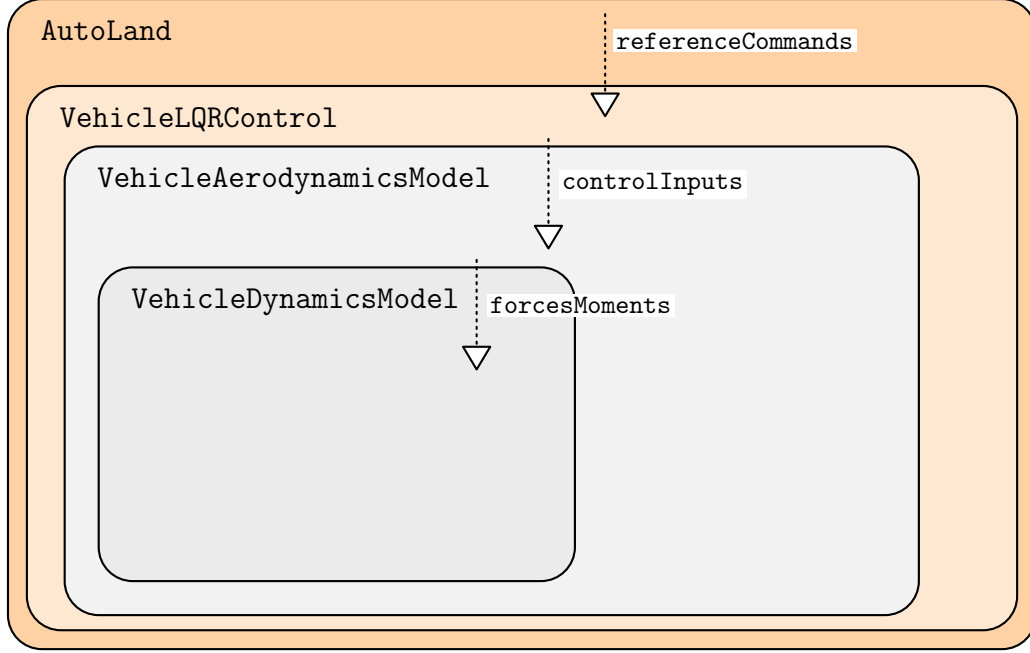


Figure 1: Nested module view for the closed-loop vehicle simulation stack.

2.2 System Architecture

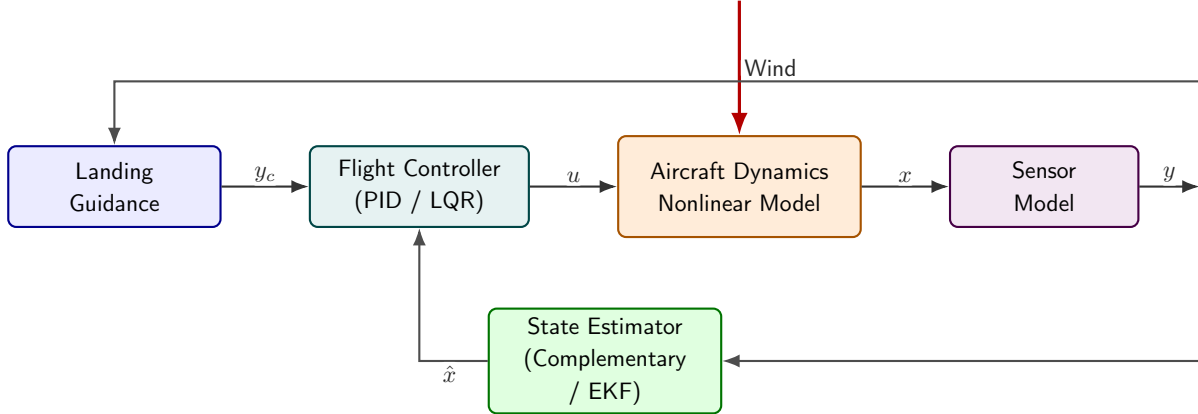


Figure 2: High-level architecture of the project.

The overall architecture follows a guidance-control-estimation pipeline. The landing-guidance module generates `referenceCommands` for commanded course, airspeed, and altitude. These commands are tracked by either the PID or LQR flight controller, which produces actuator commands for the nonlinear aircraft model. The aircraft state is passed through the sensor model to generate measurements. Those measurements are used by the state estimator, whose estimated state can be fed back to the controller. Wind enters the aircraft dynamics as an external disturbance.

2.3 Module Summary

Table 2 summarizes the top-level modules that are new or substantially modified for the final project.

Module Name	New/Modified	Purpose
LQR Controller	New	Replace the successive loop controller with LQR controller to improve the control performance.
State Estimator (Complementary Filter / EKF)	Modified	Estimate aircraft attitude, body rates, airspeed, altitude, course, and position from noisy simulated sensor measurements and provide reusable state feedback for the existing PID controller and the future LQR controller.
Landing Guidance (AutoLandGuidance)	New	Generate commanded altitude, airspeed, and course for the final approach, glide slope, flare, and touchdown phases of landing.

2.4 How the Code Will Be Run

This section follows the current `README.md` instructions for the critical-path demo. The expected grading path is the GUI-based auto-landing demonstration, launched from the repository root after setting up the Python environment and installing the required packages.

- Create and activate a Python 3.9+ environment
- Install dependencies with `pip install -r requirements.txt`
- Launch the demo from the project root with `python3 AutoLandingSim.py`
- When the ECE163 Auto Landing Simulation window opens, go to the **Controller** tab
- Leave the controller selection at the default **LQR** mode
- Leave **Use estimator for control** unchecked for the default stable demo path described in the README
- Press **Play** and monitor the **Control Response** tab while the simulation runs
- Expected output: the aircraft starts on approach, receives landing-guidance references, flies the landing profile, transitions through touchdown and rollout, and updates the GUI plots and state displays throughout the run

2.5 Expected Behavior

When the GUI demo starts, the simulator initializes the aircraft near the trim condition used for the landing approach. The `AutoLandGuidance` module then generates commanded course, altitude, and airspeed references for each landing phase. Following the behavior described in the `README.md`, the expected reference progression is:

- **APPROACH**: align with the runway heading and maintain the approach altitude
- **GLIDESLOPE**: descend along the landing path while tracking the commanded airspeed
- **FLARE**: reduce the altitude command to soften touchdown
- **TOUCHDOWN / ROLLOUT**: transition onto the runway and continue the rollout segment

In the 3D view, the aircraft should track the runway centerline and descend toward the runway. In the **Control Response** tab, course, speed, and height plots should show commanded-versus-actual behavior, and the throttle, aileron, elevator, and rudder plots should remain finite and update continuously. In the **States** tab, the full vehicle states should evolve continuously throughout the landing sequence. If the estimator path is enabled, the **Estimated States** and **Sensors** tabs should expose the corresponding estimated and noisy measurement signals for inspection.

The main pass condition for the expected behavior is the same as in the `README` critical-path demo: the GUI runs without crashing, the aircraft completes the landing sequence, and the plots continue updating throughout the run.

3. Module Details

3.1 Module A: LQR Controller

3.1.1 Purpose

The purpose of the LQR controller module is to replace the baseline successive-loop autopilot with a model-based state-space controller that can track landing references more systematically. Rather than tuning many nested PID loops independently, the LQR design uses the trim-linearized aircraft model to compute optimal feedback gains for the longitudinal and lateral channels. This gives the project a second control strategy that can be compared directly against the original controller under the same landing guidance commands.

The runtime wrapper, `VehicleLQRControl`, separates controller design from controller execution. A design function computes trim conditions, linear models, tracking gains, and optional integral-action gains. The runtime controller then uses the current aircraft state or estimated state to compute

elevator, throttle, aileron, and rudder commands, apply saturation, and pass the results into the existing aircraft model.

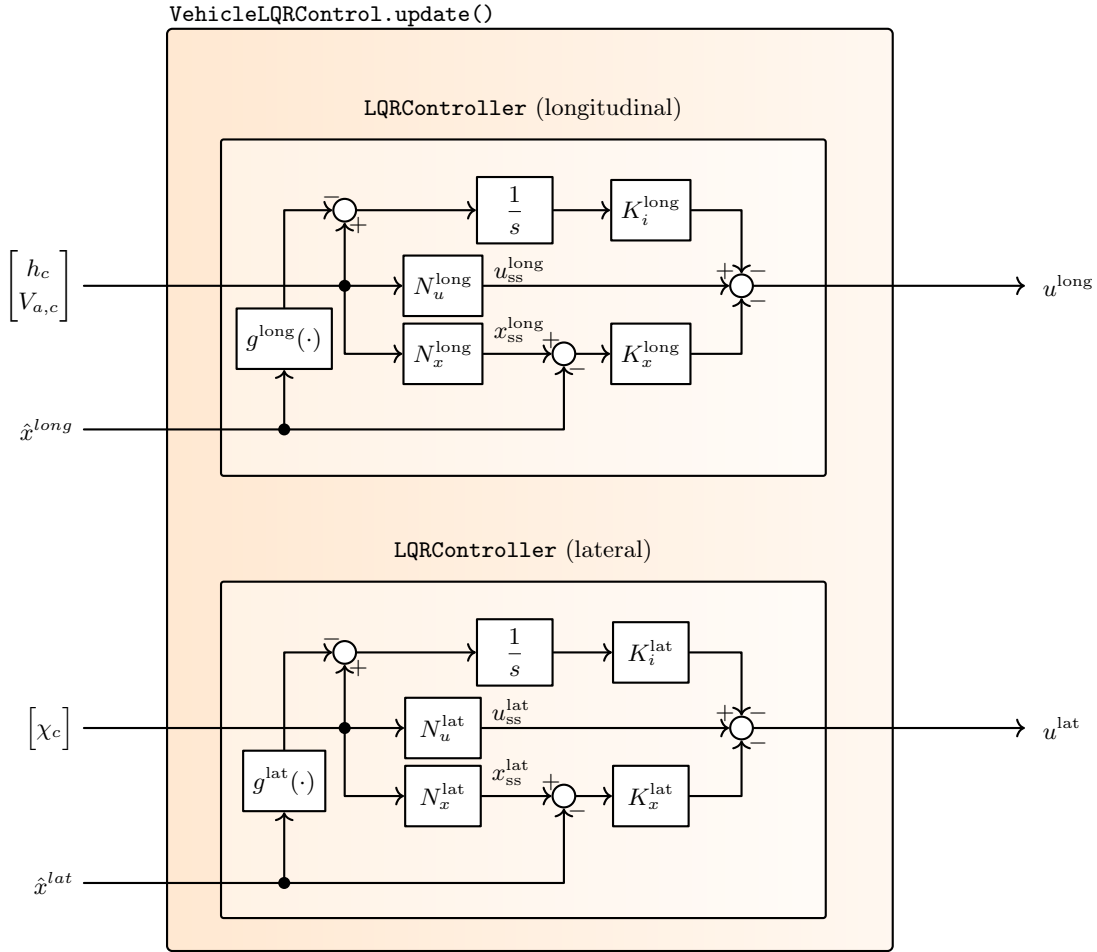


Figure 3: Internal information flow of the trim-centered LQR controller. The controller separates the longitudinal and lateral channels, computes steady-state tracking terms from the reference commands, and augments the design with integral action on output tracking errors.

3.1.2 Inputs

- Reference commands: commanded altitude h_c , commanded airspeed $V_{a,c}$, and commanded course χ_c
- Current aircraft state or controller-facing estimated state
- Trim state and trim control inputs used to build the controller bundle
- Linearized longitudinal and lateral state-space models, together with the LQR weighting matrices Q , R , and optional integral weights Q_i

3.1.3 Outputs

- Elevator and throttle commands from the longitudinal controller
- Aileron and rudder commands from the lateral controller
- Debug information including trim offsets, steady-state feedforward terms, integrator states, and saturated versus unsaturated control signals

3.1.4 Internal States / Attributes

- Controller bundle containing trim condition, longitudinal design, lateral design, and weight selections
- Two runtime `LQRController` objects: one for the longitudinal channel and one for the lateral channel
- Integral-error states for altitude, airspeed, and course tracking when integral action is enabled
- Input saturation limits, anti-windup settings, and controller debug data
- Optional sensor-model and estimator handles when the controller is configured to use estimated states

3.1.5 Main Methods

Method	Description
<code>BuildVehicleLQRController()</code>	Computes the trim condition if needed, linearizes the aircraft, forms the longitudinal and lateral LQR designs, and packages the result into a reusable controller bundle.
<code>setLQRController()</code>	Loads a controller bundle into the runtime wrapper, constructs the two channel controllers, and initializes trim and debug data.
<code>UpdateControlCommands()</code>	Extracts the controller state vectors, computes steady-state tracking terms, evaluates the LQR feedback law, and returns the current control-surface commands.
<code>update()</code>	Runs one closed-loop step by selecting the control state, computing new commands, updating the nonlinear vehicle model, and updating the sensors and estimator when enabled.

<code>reset()</code>	Resets the plant, sensors, estimator, control outputs, integrator states, and debug information to a consistent trim-centered initial condition.
----------------------	--

3.1.6 Logic / Algorithm

1. Build a trim-centered controller bundle from the desired flight condition and the linearized longitudinal and lateral aircraft models.
2. At runtime, read the current vehicle state. If estimator-assisted control is enabled, substitute the controller-facing estimated state for the measured state where appropriate.
3. Extract the reduced longitudinal and lateral state vectors used by the two LQR designs.
4. Convert the physical reference commands into the tracking outputs expected by each subsystem, then compute the steady-state trim offsets associated with those references.
5. Evaluate the longitudinal and lateral LQR control laws, including integral-action states and anti-windup logic when enabled.
6. Combine the two channel outputs into elevator, throttle, aileron, and rudder commands and send them to the existing aerodynamic model.
7. Log the internal controller signals for later tuning and PID-versus-LQR comparison.

3.1.7 Equations

The controller uses a trim-centered output-tracking LQR law. For a linearized subsystem,

$$\dot{x} = Ax + Bu, \quad y = Cx + Du \quad (1)$$

$$J = \int_0^\infty (x^T Q x + u^T R u) dt \quad (2)$$

and the continuous-time Riccati solution produces the state-feedback gain

$$u = u_{\text{eq}} + (-Kx + Fy_{\text{ref}}). \quad (3)$$

When integral action is enabled for output tracking, the controller augments the design with

$$\dot{z} = y_{\text{meas}} - y_{\text{ref}}, \quad (4)$$

so that the implemented control law becomes

$$u = u_{eq} - K_x x - K_i z + F y_{ref}. \quad (5)$$

In this project, the longitudinal controller tracks altitude and airspeed, while the lateral controller tracks course.

3.1.8 Integration with Existing Framework

This module is integrated as an alternative closed-loop controller that preserves the interfaces already used by the baseline simulator. The landing-guidance module still outputs a standard `referenceCommands` object, and the LQR wrapper still drives the existing `VehicleAerodynamicsModel`. Because of that interface compatibility, the GUI can switch between PID and LQR without changing the rest of the simulation structure. The controller also supports optional sensor and estimator hooks so that the same runtime path can be evaluated with either truth-state feedback or estimated-state feedback.

3.2 Module B: State Estimator

This module is the main contribution of Hanwen Chen. Its role is to replace truth-state feedback with realistic sensor-driven state estimation and to provide a reusable estimated-state interface for both the PID controller and the future LQR controller.

3.2.1 Purpose

The purpose of this module is to estimate the current aircraft state from noisy simulated onboard sensors rather than from perfect simulator truth data. This makes the baseline simulator more realistic and creates a consistent interface for future controller integration. The estimator is also designed to support comparison between a baseline complementary-filter method and an extended Kalman filter (EKF) method so that the team can quantify whether the EKF improves state quality enough to justify using it in the full system.

3.2.2 Inputs

- Noisy gyroscope measurements: $gyro_x$, $gyro_y$, $gyro_z$
- Noisy accelerometer measurements: $accel_x$, $accel_y$, $accel_z$
- Noisy magnetometer measurements: mag_x , mag_y , mag_z

- Barometer measurement for altitude estimation
- Pitot measurement for airspeed estimation
- GPS measurements: position, altitude, speed-over-ground, and course-over-ground
- Estimator gains and configuration parameters loaded from the simulator

3.2.3 Outputs

- Estimated attitude states: roll, pitch, yaw
- Estimated body rates: p , q , r
- Estimated navigation states: pn , pe , pd , and chi
- Estimated airspeed V_a
- A controller-facing estimated state and a GUI-facing estimated state for plotting and debugging

3.2.4 Internal States / Attributes

- Estimator mode selector (Complementary Filter or EKF)
- Attitude-estimation state and associated gains
- Airspeed-estimation state and associated gains
- Altitude-estimation state and associated gains
- Course and position-estimation states
- Current estimated vehicle-state object
- Access to the sensor model used by the simulator

3.2.5 Main Methods

Method	Description
<code>setEstimatorMode()</code>	Switches between complementary-filter mode and EKF mode for direct comparison and debugging.
<code>Update(sensorData)</code>	Performs one estimator update using the current noisy sensor packet and returns the estimated vehicle state.

<code>estimateAttitude()</code>	Updates attitude and body-rate related states using the selected estimator path.
<code>estimateAltitude()</code>	Estimates altitude from barometer and GPS measurements.
<code>estimateAirspeed()</code>	Estimates airspeed from pitot measurements and estimator dynamics.
<code>estimateCourse()</code>	Estimates course using propagated dynamics and GPS course information.
<code>estimatePosition()</code>	Updates position using GPS-derived navigation information.
<code>getRawEstimatedState()</code>	Returns the controller-facing estimate that should be used for control integration.
<code>getEstimatedState()</code>	Returns the estimate used for GUI display and comparison plots.

3.2.6 Logic / Algorithm

The estimator is organized as a set of cooperating estimation blocks rather than one single monolithic full-state filter.

1. Read the current noisy sensor packet generated by the simulator sensor model.
2. Update the attitude estimate using either the complementary-filter path or the EKF path, depending on the selected mode.
3. Update the altitude estimate using barometer and GPS information.
4. Update the airspeed estimate using pitot data.
5. Update the course and inertial position estimates using GPS speed-over-ground and course-over-ground measurements.
6. Assemble the results into a vehicle-state structure that can be consumed by the controller and visualized in the GUI.

3.2.7 Integration Notes

This module is integrated into the existing UAV simulator through the controller interface. The controller can read the estimated state rather than the truth state, which allows the simulation to behave more like a real flight-control system. The same estimator interface is intended to be reused by the LQR controller so that both PID and LQR can be evaluated under the same sensing assumptions. The current implementation is strongest on altitude, airspeed, and basic navigation states, while yaw and lateral-directional estimation remain the most challenging part of the design.

Because of this, the estimator is currently best integrated through a hybrid approach in which the most reliable estimated states are introduced first.

3.3 Module C: Landing Guidance (AutoLandGuidance)

3.3.1 Purpose

The purpose of this module is to generate reference commands that guide the UAV through an autonomous landing sequence. Commercial flights use specific traffic patterns to coordinate all flights at an airport. Similar to the Instrument Landing System (ILS), this module only simulates the final approach portion of the traffic pattern, assuming that the aircraft is already directed toward the runway environment; in commercial flight, a localizer or radio signal provides lateral guidance up to 18NM at 10° from the runway centerline.

The module computes commanded altitude, airspeed, and course values that allow the aircraft to align with the runway, descend along a glide slope, perform a flare maneuver, and complete touchdown.

This module acts as the guidance layer of the system. It converts the current aircraft state and runway parameters into reference commands that are passed to the flight control system. These commands are then tracked by either the PID controller or the LQR controller implemented in other modules.

The landing sequence is implemented as a finite state machine consisting of four phases: approach, glide slope descent, flare, and touchdown. Each phase generates different reference commands appropriate for that stage of the landing.

3.3.2 Inputs

- Current aircraft state (position, altitude, airspeed, and course)
- Runway location and heading
- Aircraft aerodynamic parameters and stall speed
- Simulation time step dT

3.3.3 Outputs

- Commanded altitude h_c
- Commanded airspeed $V_{a,c}$
- Commanded course χ_c

These outputs are packaged into a `referenceCommands` structure and passed to the flight controller.

3.3.4 Internal States / Attributes

- Current landing phase (APPROACH, GLIDESLOPE, FLARE, TOUCHDOWN)
- Simulation time
- Flare start time
- Flare start altitude
- Glide slope angle
- Runway position and heading
- Estimated stall speed of the aircraft

3.3.5 Main Methods

Method	Description
<code>initialize()</code>	Initializes the landing guidance module and sets the initial phase to APPROACH.
<code>update(state)</code>	Computes the commanded altitude, airspeed, and course based on the current aircraft state and landing phase. It also performs phase transitions when conditions are met.
<code>reset()</code>	Resets the landing module to its initial state and restarts the landing sequence.

3.3.6 Logic / Algorithm

The autonomous landing guidance module operates using a finite state machine that transitions through four landing phases.

1. Read the current aircraft state including position, altitude, airspeed, and course.
2. Determine the aircraft's relative position to the runway.
3. Depending on the current landing phase, generate the appropriate reference commands:
 - **Approach Phase:** The aircraft aligns with the runway heading and maintains a constant approach altitude and airspeed until it reaches the glide slope intercept point.

- **Glide Slope Phase:** The aircraft descends toward the runway along a constant glide slope angle while maintaining a constant airspeed.
 - **Flare Phase:** Near the runway, the descent rate is gradually reduced to allow a smooth touchdown.
 - **Touchdown Phase:** The aircraft levels near the runway altitude while maintaining runway alignment.
4. The commanded altitude, airspeed, and course are packaged into reference commands.
 5. The commands are passed to the flight control system for tracking.

3.3.7 Equations

The glide slope descent rate is computed using the relationship between airspeed and flight path angle:

$$\dot{h} = -V_A \sin(\gamma) \quad (6)$$

where:

- V_a is the aircraft airspeed
- γ is the glide slope angle

The commanded altitude is updated by integrating the descent rate:

$$h_c(t+1) = h_c(t) + \dot{h} dT \quad (7)$$

During the flare maneuver, the altitude follows an exponential decay model to reduce the sink rate smoothly:

$$h(t) = h_{\text{flare}} e^{-t/\tau} \quad (8)$$

where:

- h_{flare} is the altitude where the flare begins
- τ is the flare time constant

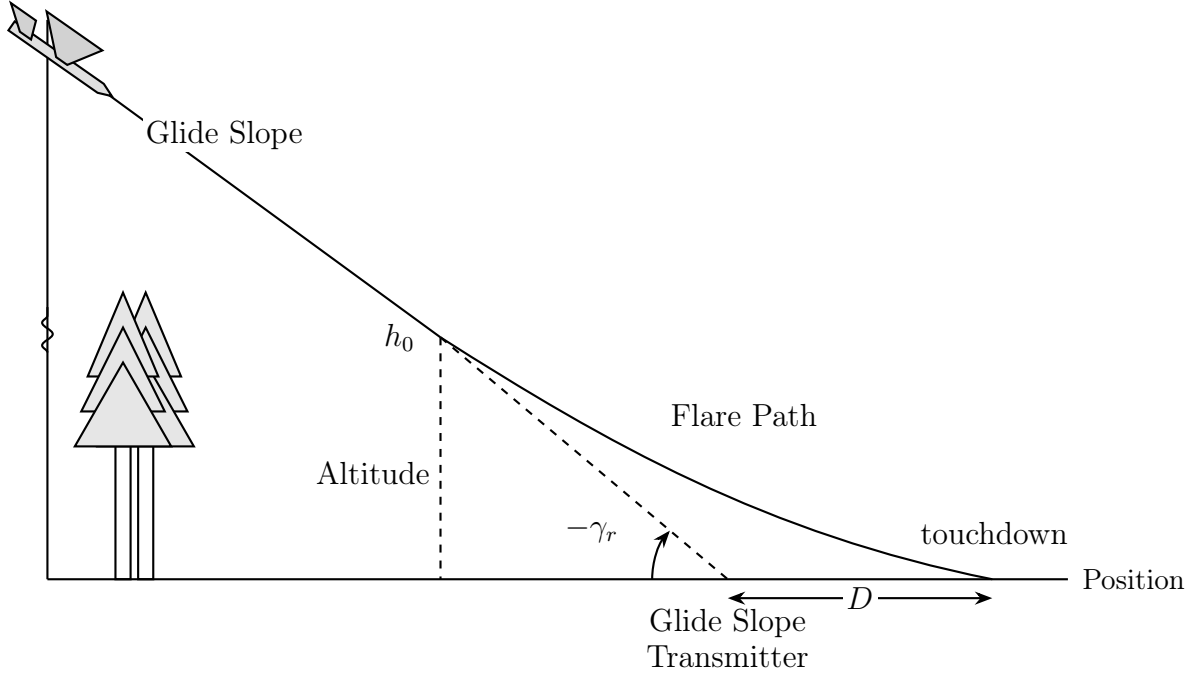


Figure 4: Reference trajectory geometry for auto-landing.

3.3.8 Reference Trajectory

3.3.9 Integration with Existing Framework

The autonomous landing guidance module integrates with the existing UAV simulator by producing reference commands that are compatible with the baseline autopilot interface. The output reference commands are consumed by the flight controller, which may be either the existing PID controller or the LQR controller developed in this project.

The module operates independently of the control implementation, allowing different control strategies to be tested while using the same landing guidance logic. In addition, the module uses state information provided by the simulator or the Kalman filter state estimator to compute landing commands.

This modular design allows the guidance, control, and estimation components of the system to remain loosely coupled while working together to achieve autonomous landing.

4. Development Plan

4.1 Task Breakdown

List the main implementation tasks and assign responsibility if relevant.

Task	Owner	Priority	Notes
LQR controller	Xianle	High	this week
Reference Generation	Gavin	High	3/15
Kalman Filter	Hanwen	High	this week
Estimator integration with controller pipeline	Hanwen	High	connect estimated-state interface to the simulator and support reuse in later LQR integration
Estimator validation scripts / test harnesses	Hanwen	High	EKF vs complementary comparison and full-system validation
Estimator result visualizations	Hanwen	High	prepare true-vs-estimated plots and presentation figures
Simulator for landing (GUI for runway)	Gavin	High	3/15
Simulator for landing (a new script for landing)	Xianle	High	this week
Comparison for results	Hanwen	High	next week

5. Testing Strategy

5.1 Testing Philosophy

Explain your testing approach:

- Validate the estimator independently before judging the controller
- Compare the EKF directly against the complementary-filter baseline under the same sensor-noise conditions
- Use both nominal-flight and landing scenarios so estimator performance is not judged from only one operating condition
- Report quantitative metrics such as RMS state-estimation error, tracking error, and control activity

5.2 Incremental Testing Plan

Test Stage	Description	Expected Result
Stage 1	Module-level sanity check	No runtime errors
Stage 2	EKF vs complementary-filter comparison on the same trajectory	EKF improves the main estimated states used by the controller
Stage 3	Closed-loop simulator test with estimator-enabled controller	Data flows correctly and the UAV remains stable enough for evaluation
Stage 4	Autonomous landing integration test	Guidance, controller, and estimator run together without runtime failure and produce comparison plots

5.3 Evaluation Scenarios

Describe specific simulation scenarios.

5.3.1 Scenario 1: Nominal Case

- Initial condition: straight and level flight near the nominal trim point with no wind
- Command / target: constant altitude, airspeed, and course
- Expected outcome: EKF and complementary filter both remain bounded, and the EKF shows improved tracking of airspeed, altitude, and attitude states

5.3.2 Scenario 2: Disturbance Case

- Disturbance: sensor noise plus changes in commanded altitude, airspeed, and course
- Expected outcome: the estimator remains stable and the resulting controller behavior stays usable for comparison

5.3.3 Scenario 3: Stress Test

- Challenging setup: autonomous landing sequence including final approach, glide slope, flare, and touchdown
- Expected outcome: both the complementary-filter baseline and the EKF version can be run and compared on the same landing scenario

5.4 Planned Outputs

List the plots, tables, and logs you expect to generate.

- True-state versus estimated-state plots for roll, pitch, yaw, p , q , r , altitude, airspeed, and course
- EKF vs complementary-filter comparison plots without landing
- Complementary-filter landing plots
- EKF landing plots
- Control response plots showing course, height, speed, pitch, roll, and control inputs
- Summary tables of RMS estimation and tracking metrics

5.5 Final Success Criteria

The estimator portion of the project will be considered successful if it runs reliably inside the simulator, remains bounded under noisy measurements, improves important estimated states relative to the complementary-filter baseline, and can be integrated into the closed-loop pipeline without immediate instability. In addition, the project must provide repeatable test harnesses and clear result visualizations for the three planned comparisons: EKF vs. complementary filter without landing, complementary filter with landing, and EKF with landing.

6. Conclusion

This design document presents the proposed architecture, modules, development plan, and testing strategy for Airplane Auto Landing System. The project extends the baseline ECE163 simulator by adding autonomous landing guidance, a trim-centered LQR flight controller, and an estimator path that supports both complementary filtering and EKF-based state reconstruction. The design follows the modular layering recommended in the course project description and emphasizes measurable success criteria, isolated module testing, and end-to-end comparison in the full landing scenario. Successful completion of the project will demonstrate that the UAV can execute a repeatable autonomous landing sequence while also providing meaningful PID-versus-LQR and complementary-versus-EKF performance comparisons.

References

1. R. W. Beard and T. W. McLain, "Small Unmanned Aircraft: Theory and Practice," Princeton University Press, 2012.

2. G. H. Elkaim, “ECE163 Final Project Description,” University of California, Santa Cruz, Winter 2026.
3. ECE163 course notes, provided simulator codebase, and accompanying module documentation used throughout the quarter.